

Apunte – Tipos de datos en C, formatos y casteo

1. ¿Qué es un tipo de dato?

Un **tipo de dato** determina:

- Qué clase de información puede almacenar una variable.
- Cuánta memoria ocupa.
- Qué operaciones se pueden realizar sobre ella.

Ejemplos:

```
int edad = 20;  
float altura = 1.80;  
char inicial = 'F';
```

En este ejemplo:

- **edad** almacena un número entero.
 - **altura** almacena un número decimal.
 - **inicial** almacena un único carácter.
-

2. Tipos de datos básicos

int

Almacena números enteros.

```
int edad = 25;
```

Ejemplos: 0, 15, -30 o 2500

Formato:

```
printf("%d", edad);  
scanf("%d", &edad);
```

float

Almacena números con decimales.

```
float altura = 1.75;
```

Ejemplos: 3.14, 0.5 o -12.7

Formato:

```
printf("%f", altura);  
scanf("%f", &altura);
```

Se puede limitar la cantidad de decimales: `printf("%.2f", altura);`

Salida: 1.75

double

También almacena números decimales, pero con mucha mayor precisión que `float`.

```
double pi = 3.14159265358979;
```

Formato:

```
printf("%lf", pi);  
scanf("%lf", &pi);
```

char

Guarda un único carácter.

```
char letra = 'A';
```

Siempre se escribe entre comillas simples.

Formato:

```
printf("%c", letra);
scanf(" %c", &letra);
```

Importante:

Observá el espacio antes de `%c`.

```
scanf(" %c", &letra);
```

Ese espacio hace que `scanf` ignore los Enter (`\n`) que puedan haber quedado pendientes en el teclado.

3. Tabla de formatos más utilizados

Tipo	printf	scanf
int	%d	%d
float	%f	%f
double	%lf	%lf
char	%c	%c

4. ¿Por qué existen estos símbolos?

`printf` y `scanf` no saben qué tipo de variable les estamos pasando.

Por ejemplo: `int edad = 20;`

```
printf("%d", edad);
```

El `%d` le indica a `printf`: "Lo que viene es un entero."

Si escribiéramos: `printf("%f", edad);`

el programa interpretaría la memoria como si fuera un `float`, obteniendo un resultado incorrecto (comportamiento indefinido). Lo mismo ocurre con `scanf`.

```
scanf("%d", &edad);
```

Le indica: "Leé un entero y guardalo en esta dirección de memoria."

5. El operador & en scanf

Una duda muy común. En `printf` escribimos: `printf("%d", edad);`

En cambio, en `scanf`: `scanf("%d", &edad);`

¿Por qué? Porque:

- `printf` **lee** el valor.
- `scanf` **modifica** la variable.

Para poder modificarla necesita conocer dónde está almacenada en memoria. El operador `&` significa: "Dirección de memoria de esta variable."

Ejemplo:

```
int edad;
```

```
scanf("%d", &edad);
```

El usuario escribe: 23. Entonces la variable pasa a contener: `edad = 23;`

6. Conversión automática de tipos

C muchas veces convierte tipos automáticamente. Ejemplo:

```
int a = 5;
```

```
float b = 2.0;
```

```
float resultado = a + b;
```

Aunque `a` es entero, C lo transforma automáticamente en: 5.0

Entonces realiza: $5.0 + 2.0 = 7.0$

Esto se llama **conversión implícita**.

7. ¿Qué es el casteo?

El **casteo** (casting) consiste en convertir un dato de un tipo a otro de forma explícita.

La sintaxis es: (tipo) variable

Ejemplo: (float) numero

8. ¿Por qué se usa el casteo?

Principalmente para evitar errores cuando se mezclan distintos tipos de datos. El caso más importante es la división entre enteros.

Ejemplo sin casteo

```
int a = 5;
```

```
int b = 2;
```

```
float resultado;
```

```
resultado = a / b;
```

Muchos esperan: 2.5

Pero el resultado es: 2.0

¿Por qué? Porque ambos operandos son enteros.

Entonces C hace: $5 / 2 = 2$

Recién después guarda ese 2 dentro del **float**.

Ejemplo con casteo

```
resultado = (float)a / b;
```

Ahora ocurre esto: $5.0 / 2$

Resultado: 2.5

También podría hacerse: **resultado = a / (float)b;** o **resultado = (float)a / (float)b;**

Los tres producen el mismo resultado.

9. Otro ejemplo útil

Supongamos que queremos calcular un promedio.

```
int nota1 = 8;  
int nota2 = 9;
```

```
float promedio;
```

Si hacemos: promedio = (nota1 + nota2) / 2;

Obtendremos: 8.0

Porque: 17 / 2 = 8

La parte decimal se pierde antes de asignar el resultado al **float**. La forma correcta es:
promedio = (float)(nota1 + nota2) / 2;

Ahora el cálculo es: 17.0 / 2

Resultado: 8.5

10. Conversión de float a int

También puede convertirse un decimal en entero.

```
float numero = 7.9;
```

```
int entero;
```

```
entero = (int) numero;
```

Resultado: 7

Importante: No redondea. Simplemente elimina la parte decimal.

Ejemplos:

7.9 → 7

3.2 → 3

5.999 → 5

11. Errores comunes

Error 1

```
float promedio;
```

```
promedio = 5 / 2;
```

Resultado: 2.0

Correcto: promedio = (float)5 / 2;

Error 2

```
scanf("%f", &edad);
```

si **edad** es un **int**. Cada tipo de dato debe utilizar el especificador correspondiente.

Error 3

```
printf("%d", altura);
```

si **altura** es un **float**. Debe usarse: printf("%f", altura);

12. Resumen

Tipo	Ejemplo	printf	scanf
int	25	%d	%d
float	3.14	%f	%f
double	3.141592	%lf	%lf
char	'A'	%c	%c

Regla de oro del casteo

Siempre que un cálculo con enteros deba producir un resultado decimal (por ejemplo, un promedio), convertí **al menos uno de los operandos** a **float** o **double** antes de realizar la operación.

```
promedio = (float)suma / cantidad;
```

Es una práctica muy común y evita uno de los errores más frecuentes al comenzar a programar en C.